

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE:CSA06

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1,BL2,BL3) Assessment Tool Weightage: 20%

QUESTIONS: 10

Total Marks: 50

Annexure A

Scenario Based Assessment

Code of Conduct:

I _Sasikumar Megha__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	CSA BS	Date:	3/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

90
 —
 100

Code of Conduct

I, Sasikumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	/	/		
Application of Concepts	30%	/	/		
Analytical & Decision-Making Skills	20%	/	/		
Solution Design & Approach	15%	/	/		
Clarity, Justification & Presentation	10%	/	/		
Total Marks (100):		90/100			

Faculty In-charge	Course Coordinator	Head of Department
Name: <u>[Signature]</u>	Name: _____	Name: _____
Signature: <u>[Signature]</u>	Signature: _____	Signature: _____
Date: <u>3/8/26</u>	Date: _____	Date: _____

Annexure A

Scenario-Based Assessment Q91.

Smart Traffic Control System – Selection Sort vs Bubble Sort

a) Analyze how Selection Sort and Bubble Sort would perform on this dataset.

Selection Sort Execution

Suppose the vehicle counts collected are:

45, 20, 35, 10, 50 Pass 1

- Find the smallest element (10).
- Swap it with the first element.
- Result: **10, 20, 35, 45, 50**

Pass 2

- Find the smallest element from remaining elements.
- 20 is already the smallest.
- Result: **10, 20, 35, 45, 50**

Pass 3

- Find the smallest element among 35, 45, 50.
- No swap required.

Pass 4

- Compare 45 and 50.
- Already sorted.

Final Sorted List 10, 20, 35, 45, 50

Bubble Sort Execution

Initial List:

45, 20, 35, 10, 50 Pass 1

- $45 > 20 \rightarrow$ Swap
- $45 > 35 \rightarrow$ Swap
- $45 > 10 \rightarrow$ Swap
- $45 < 50 \rightarrow$ No swap

Result:

20, 35, 10, 45, 50

Pass 2

- $20 < 35$
- $35 > 10 \rightarrow \text{Swap}$
- $35 < 45$

Result:

20, 10, 35, 45, 50

Pass 3

- $20 > 10 \rightarrow \text{Swap}$
- Remaining already sorted.

Result:

10, 20, 35, 45, 50

Performance

- Selection Sort performs fewer swaps.
- Bubble Sort performs many adjacent swaps.
- Both require multiple comparisons.

b) Compare their efficiency

Parameter	Selection Sort	Bubble Sort
Best Case	$O(n^2)$	$O(n)$ (Optimized)
Average Case	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$
Parameter	Selection Sort	Bubble Sort
Comparisons	$n(n-1)/2$	$n(n-1)/2$

Swaps	$O(n)$	$O(n^2)$
Memory	$O(1)$	$O(1)$

c) Recommendation

Selection Sort is recommended because:

- Performs fewer swaps.
- Better when swap operations are costly.
- Easy to implement.
- Suitable for small datasets like vehicle counts collected from a few junctions.

Q92. Drone Delivery System – Brute Force Closest Pair

a) Explain how the brute-force closest pair algorithm works.

Assume warehouse locations are

A(2,3), B(4,5), C(8,6), D(1,7) Step 1

Calculate distance between every pair.

Distance Formula

$$[d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}]$$

Step 2

Calculate distances.

- AB
- AC
- AD
- BC
- BD
- CD

Example:

AB

$$[\sqrt{(4-2)^2 + (5-3)^2}]$$

$=\sqrt{4}+\sqrt{4}$
 $=\sqrt{8}$
 $=2.83$
]

Similarly compute all distances.

Suppose

Pair	Distance
AB	2.83
AC	6.71
AD	4.12
BC	4.12
BD	3.60
CD	7.07

The minimum distance is **2.83**.

Hence, warehouses **A and B** are the closest.

b) Time Complexity

For **n** locations,

Every point is compared with every other point.

Number of comparisons **$n(n-1)/2$**

Therefore,

Time Complexity = $O(n^2)$

Space Complexity = **$O(1)$**

c) Scalability Issues

- Computation increases rapidly.
- Doubling locations almost quadruples computations.
- Slow for thousands of warehouses.
- Not suitable for real-time drone routing.

Q93. University Admission System – Linear Search

a) Explain how Linear Search works. Applicant

IDs

A101 A102 A103 A104 A105

Search for **A104**

Execution

Compare

A101 → No

A102 → No

A103 → No

A104 → Found

Number of comparisons = **4**

If searching for A105

Need **5 comparisons**. If

applicant does not exist

Entire list is searched.

b) Performance Analysis

Best Case

Applicant found at first position.

Comparisons = 1

Time = **$O(1)$**

Average Case

Applicant found in middle.

Comparisons $\approx n/2$

Time = **$O(n)$**

Worst Case

Applicant at last position or absent.

Comparisons = n

Time = $O(n)$

c) Would Sorting Improve Efficiency?

Yes.

After sorting, Instead of

Linear Search,

Binary Search can be used.

Comparison

Search Method Complexity

Linear Search $O(n)$

Binary Search $O(\log n)$

Hence sorting significantly improves retrieval when searches are frequent.

Q94. Disaster Response System – Brute Force Convex Hull

a) Explain how brute-force convex hull operates.

Suppose affected zones are represented by

A, B, C, D, E

Execution

Step 1

Take line AB.

Check whether all remaining points lie on one side.

If yes,

AB belongs to convex hull.

Otherwise reject.

Step 2

Repeat for

AC

AD

AE BC

BD

...

Continue until every pair is checked.

Finally,

Boundary points form the convex hull.

b) Computational Complexity

For every pair $O(n^2)$

Checking remaining points $O(n)$

Overall $O(n^3)$

Space Complexity $O(1)$

c) Limitations

- Large number of point comparisons.
- High execution time.
- Poor performance for GIS applications.
- Unsuitable for real-time disaster monitoring.
- Modern algorithms like Graham Scan and Jarvis March are much faster.

Q95. Cybersecurity Monitoring – Brute Force String Matching

a) Explain how the algorithm works.

Text (Network Log)

ABABABCABAB

Pattern

ABC

Execution

Compare pattern from first position.

ABA

ABC

Mismatch.

Shift one position.

BAB

ABC

Mismatch.

Shift again.

ABA

ABC

Mismatch.

Shift again.

ABC

ABC

Complete match found.

Continue until end of text.

b) Time Complexity

Let

- Text length = n
- Pattern length = m

Worst case	$O(nxm)$
Best case	$O(n)$
Space complexity	$O(1)$

c) Inefficiency for Large Logs

- Every possible position is checked.
- Many repeated character comparisons.
- Processing millions of log entries becomes slow.
- High CPU usage.
- Not suitable for real-time intrusion detection.
- Algorithms such as **Knuth–Morris–Pratt (KMP)** and **Boyer–Moore** avoid unnecessary comparisons and provide much better performance for large-scale network logs.